

Computer programming: an overview

FUNDAMENTALS OF PROGRAMMING

Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

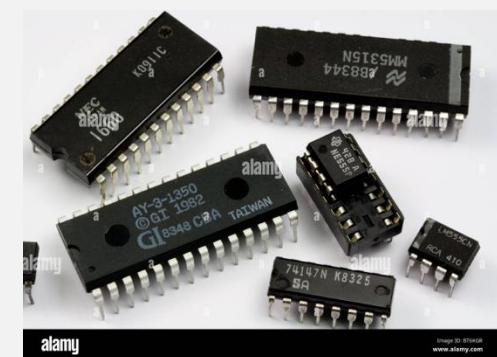
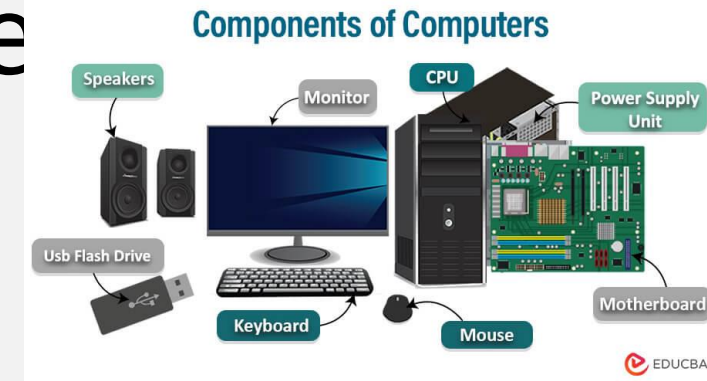
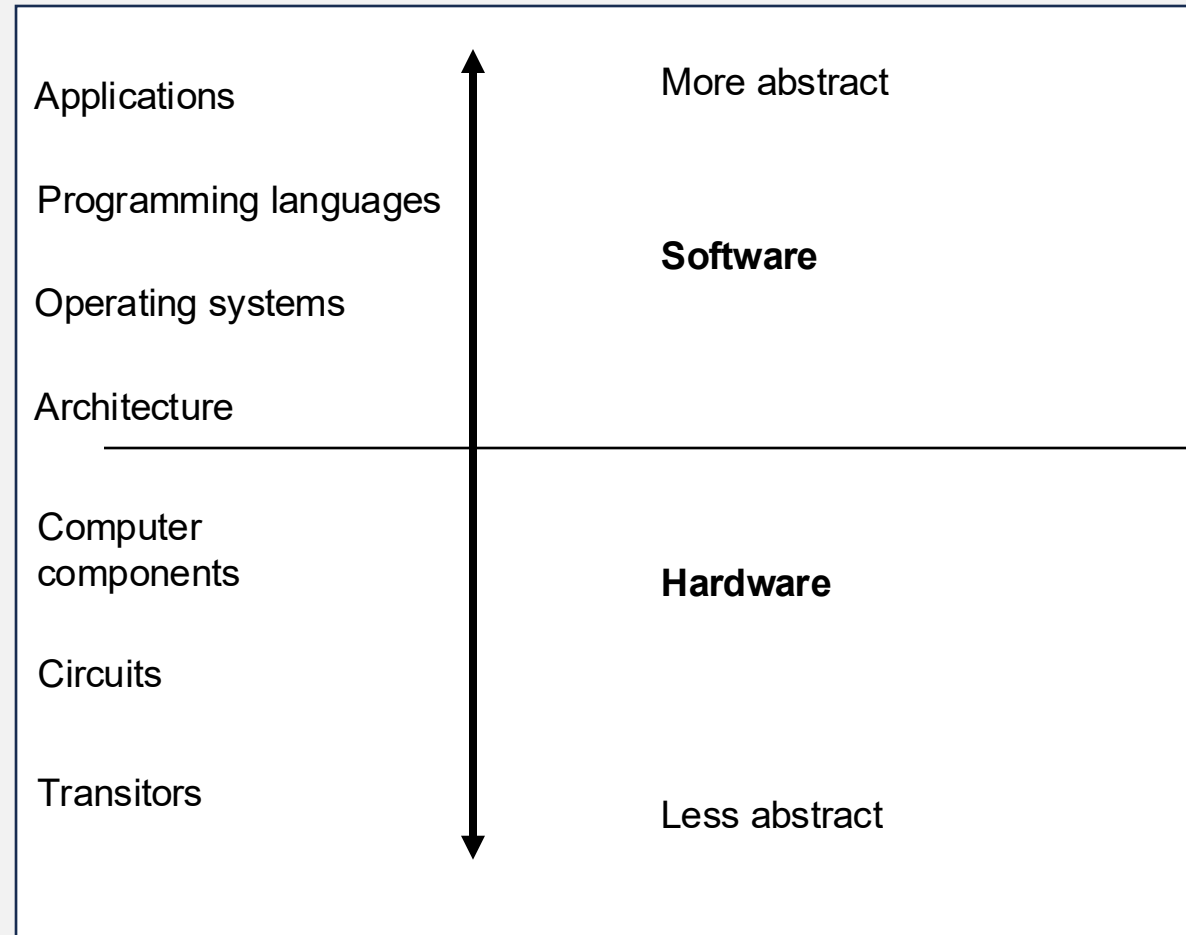
Plan for today

- Introduction to Computers
- Introduction to Algorithms
- Exercises

Plan for today

- **Introduction to Computers**
- Introduction to Algorithms
- Exercises

Introduction to Computer



Applications

- Any program you run on your computer
 - Microsoft Word
 - Google Chrome
 - Note application
- .exe (Windows) or .app (Mac) file
- Can use without knowing how it works behind the scenes (**abstraction**)
- Programs are written in **code**, the next layer of abstraction

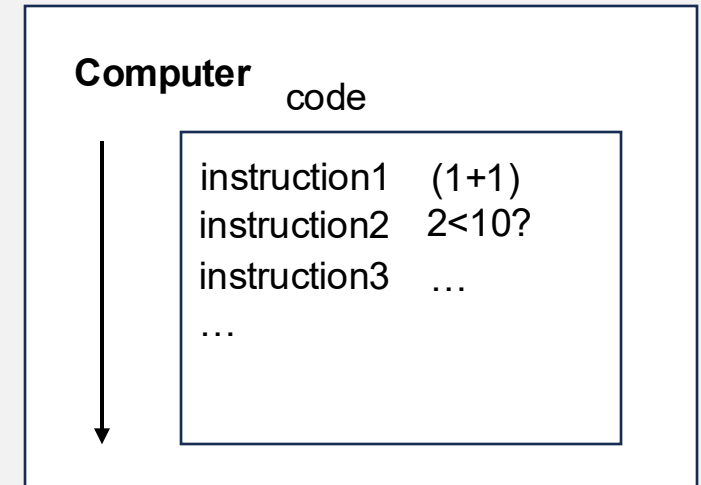
Introduction to Code

- **Strengths of Computers**
 - Very fast (billions of operations per second)
 - Very accurate
- **Weaknesses of Computers**
 - Literal
 - Only simple operations
 - Addition
 - Comparsion
 - Equality
 - No insight

Introduction to Code:

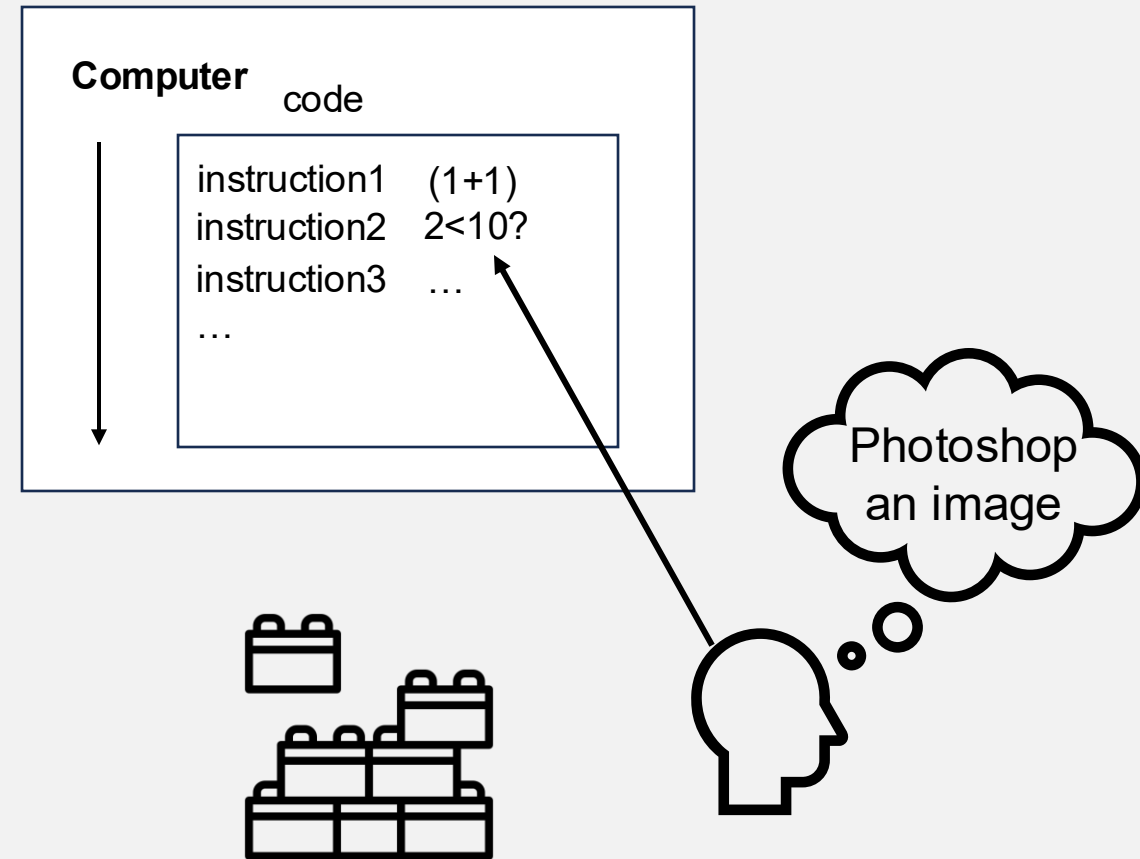
How computers work

- **Code** is a series of instructions
- Each instruction is an operation
- Computer follows each instruction, one after another
- Programs are comprised of millions of instructions



Introduction to Code: The programmer

- Programmers convert human ideas into computer code → Develop algorithms
- Code is like Legos - small bricks to big structures → Patience and stay calm!
- Computers do exactly what you tell them (no interpretive power)



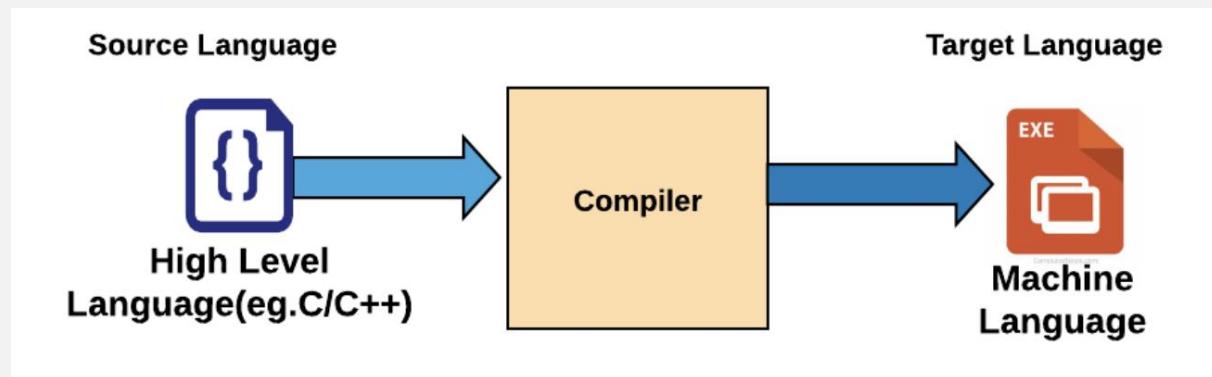
Programming Languages

- Code is written in a **programming language (PL)**
- Gives the "rules" (syntax and commands) for the code
- Each PL has strengths and weaknesses and specializations (just like human languages)
- High-level PL (closer to how humans think) and low-level PL (closer to computers, but much faster)

Types of Programming Languages

- **Compiled**

- "Source code" (what the programmer writes) is converted to "object code" (what the computer reads)
- Compiled once, run many times (like translating a book to another language)



Types of Programming Languages

- **Compiled**

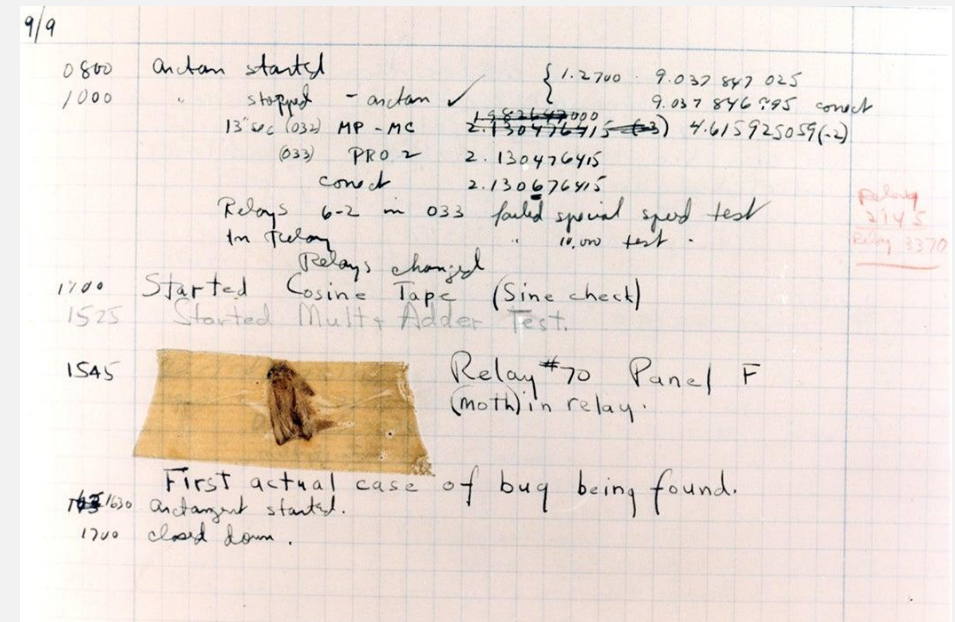
- "Source code" (what the programmer writes) is converted to "object code" (what the computer reads)
- Compiled once, run many times (like translating a book to another language)

- **Interpreted**

- Machine code instructions generated on the fly (like acting as a verbal translator in a conversation)
- Slower to run, but no compile step!

Bugs/debug

- Story: named after a moth
- Two types: **syntax** and **semantic** errors
 - Wrong “rules”
 - Results do not same as expected ones
- Compile-time error
- Run-time error



Syntax errors

- English

"cat dog boy" → not syntactically valid

"cat hugs boy" → syntactically valid

- Programming language

"hi"5 → not syntactically valid

3.2*5 → syntactically valid

Semantic errors

- Natural languages can have many meanings in a sentence

In English:

“John saw Jane driving down the street.”

In Vietnamese:

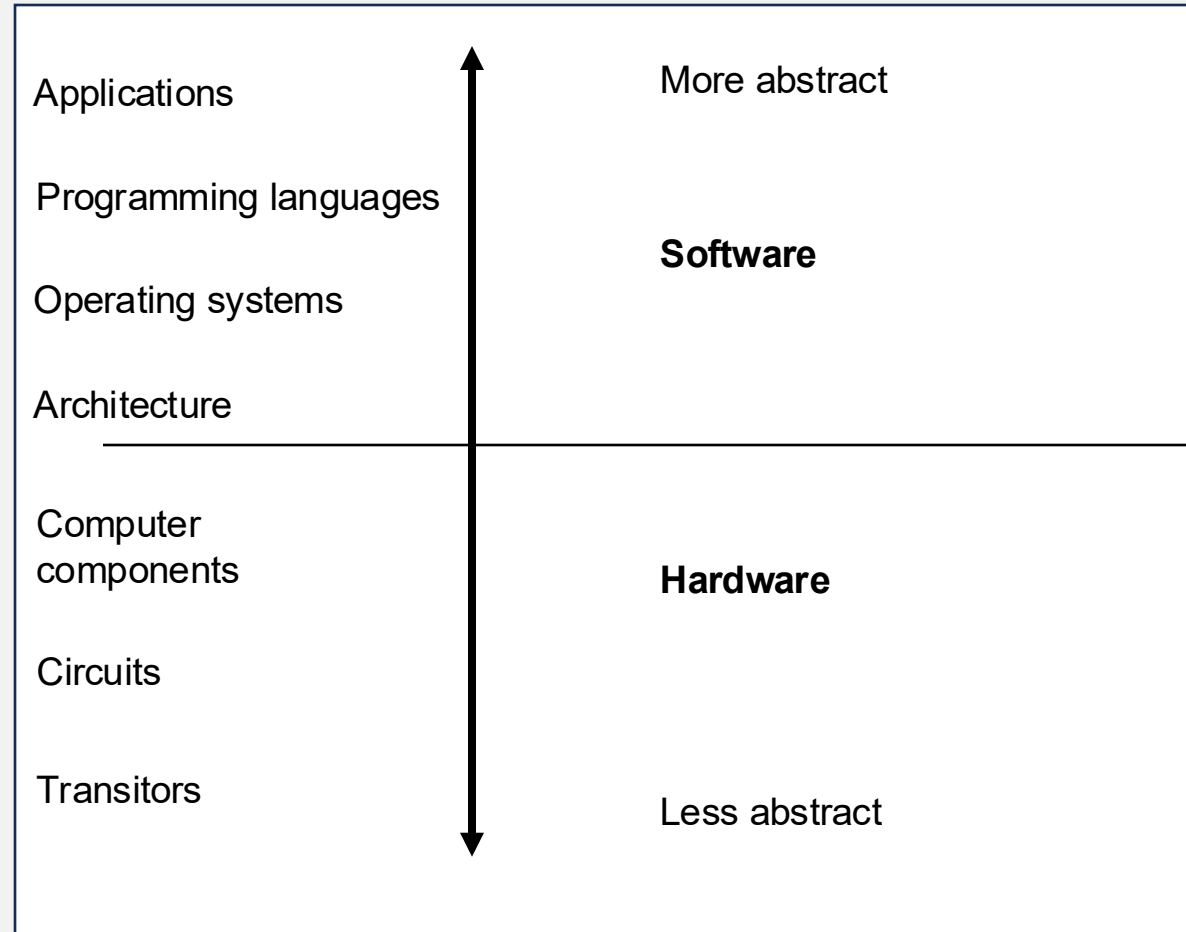
“Ông già đi nhanh quá!”

- Programming languages: have only one meaning but may not be what programmer intended
- In C++,

```
if (x % 2 == 1) cout << "x is an odd number"
```

It is correct?

Look back



The Operating System

- Starts running when the computer "boots up"
- Manager/supervisor
 - Starting/runs/stopping programs
 - Manages all shared resources (like **memory**)
 - **No matter how a program ends, all resources must be released**
- Can run multiple programs at once ("sandboxing")
- Allows a computer to change over time (updates)
- Other responsibilities:
 - File system
 - Manages windows
 - ...

Lifecycle of a program

- Load the program file from the file system
- Each program gets its own **memory**
 - Program's code
 - Data the program manipulates (like file being edited)
- OS "fetches" an instruction, which is then run (OS alternates instructions between programs)
 - Begins with the first line of code
 - Instructions like "return to step 5" keep the program running
- At the end of the program, OS needs to stop fetching instructions and ensure program returns memory

Ending programs

- Normal exit: user quits the program (e.g., by pressing "X")
- Abnormal exit: Program "not responding"
 - Use Activity Monitor (Mac)/Task Manager (Windows)
- "Illegal" activity: A program tries to access the memory of another program
- Out of Memory: A program requests more memory (e.g., you edit a lot of videos at once)

Architecture: Machine Code

Architecture refers to the instructions sent by software that hardware can understand.

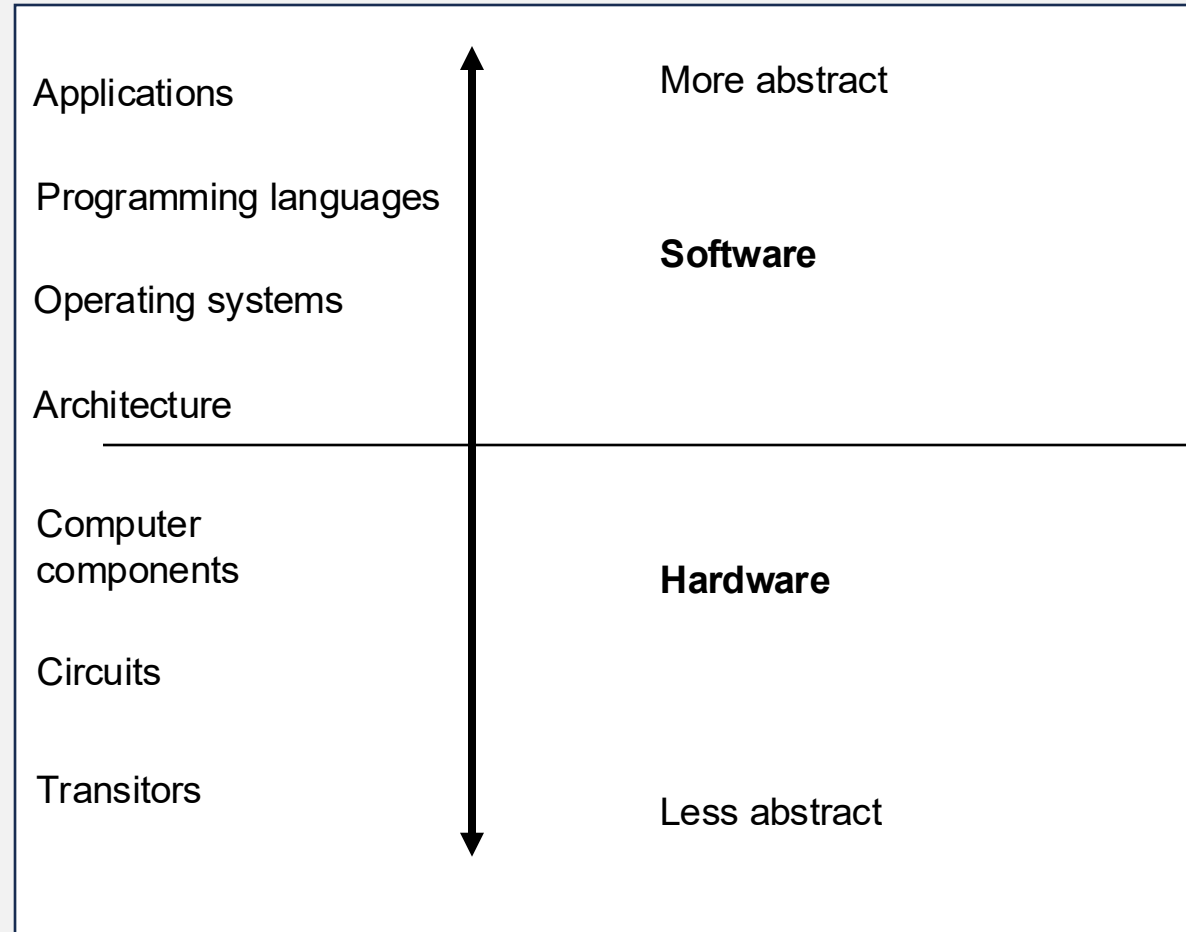
- Reminder: computer runs code instructions
- Each line of machine code is one operation
- C++ is a higher-level language
 - We have to translate C++ to machine code
- Because of how hardware works, machine code is written in **Binary** (0s and 1s)
- Machine-specific (depend on machine architecture)

Architecture: Assembly Language

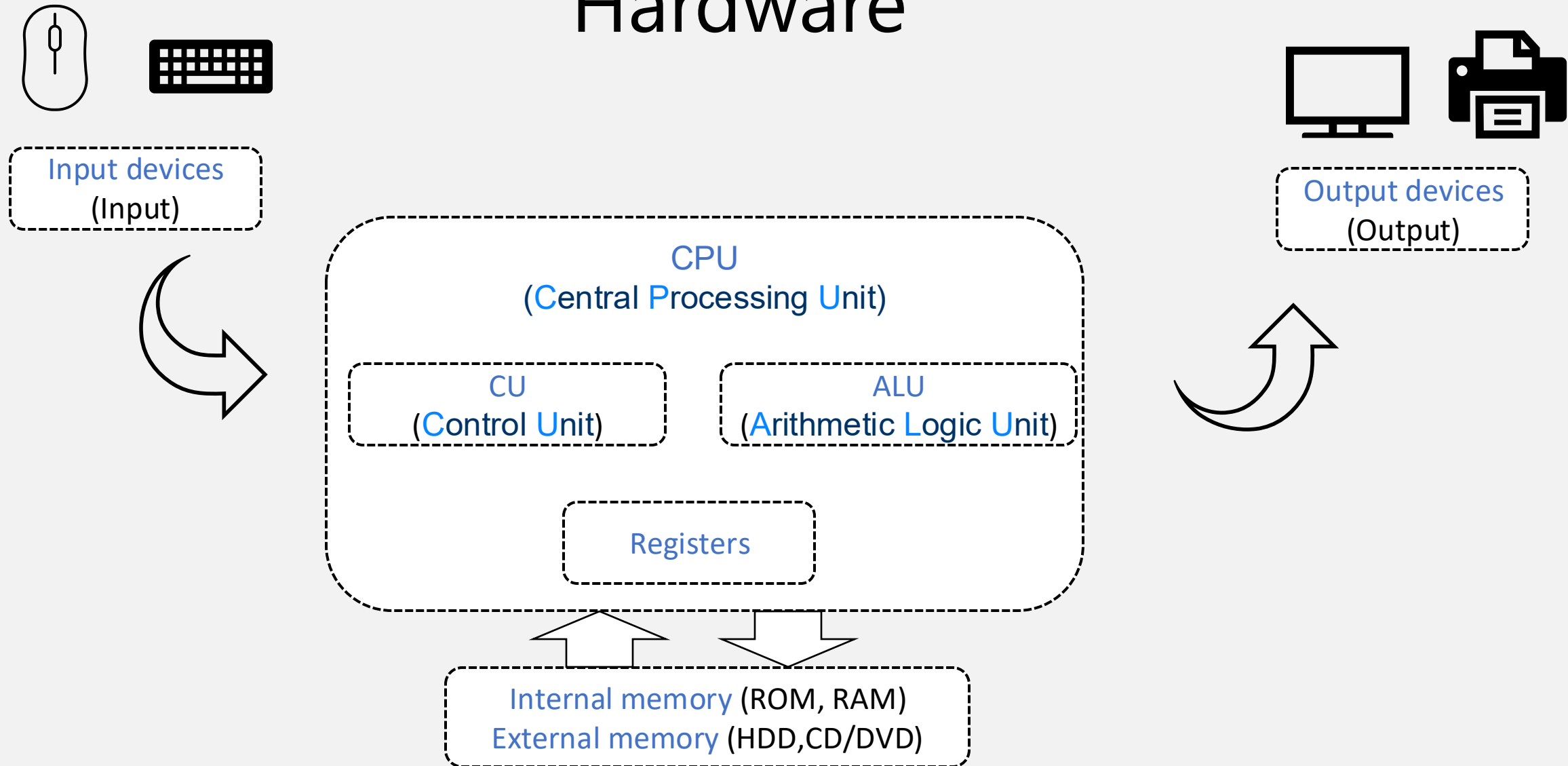
```
mov    %esp, %ebp
sub    $0x28, %esp
mov    0x804d300, %eax
add    $0x1, %eax
mov    %eax, 0x804d300
mov    0x804d300, %eax
cmp    0x8(%ebp), %eax
```

- Example
- Assembly language is converted to machine code
- %esp, %ebp, etc. are memory

Look again



Hardware



Plan for today

- Introduction to Computers
- **Introduction to Algorithms**
- Exercises

Types of Programming

- **Declarative programming**
 - Focus on results that programmers want (WHAT)
 - No matter how computer works
- **Imperative programming**
 - Focus on steps to solve problems (HOW)
 - A recipe or “how-to”

Types of Programming (example)

How to reserve the restaurant?

- **Declarative programming** perspective
 - Call a server and make a reservation
- **Imperative programming** perspective
 - Step1: Come to the restaurant
 - Step2: Look around and find a blank place
 - Step3: Move to the blank place

A numerical example

- Square root of a number x is y such that $y*y = x$
- Recipe for deducing square root of a number x (16)
 - 1) Start a **guess**, g
 - 2) If $g*g$ **closes enough** to x , stop and say g is the answer
 - 3) Otherwise, make a **new guess** by averaging g and x/g
 - 4) Using a new guess, **repeat** process until close enough (close to x)

g	$g*g$	x/g	$(g+x/g)/2$
3	9	16/3	4.17
4.17	17.36	3.837	4.0035
4.0035	16.0277	3.997	4.000002

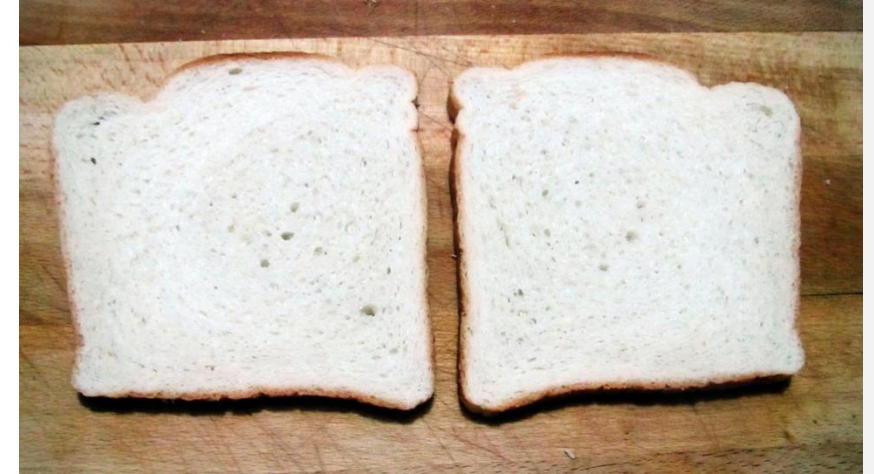
What is a recipe

- 1) Sequence of simple **steps**
 - 2) **Flow of control** process that specifies when each step is executed
 - 3) A means of determining **when to stop**
- 1 + 2 + 3 = an **algorithm**

Some characteristics of algorithms

- **Well-defined inputs, outputs:** Well-defined input, output for computers
- **Finiteness:** a finite number of steps
- **Language independence:** return same results from different programmers or programming languages
- **Feasibility:** return in a reasonable time

Write instructions to make a Peanut butter and Jelly sandwich



sources: [1](#), [2](#), [3](#), [4](#)

Write instructions to make a Peanut butter and Jelly sandwich



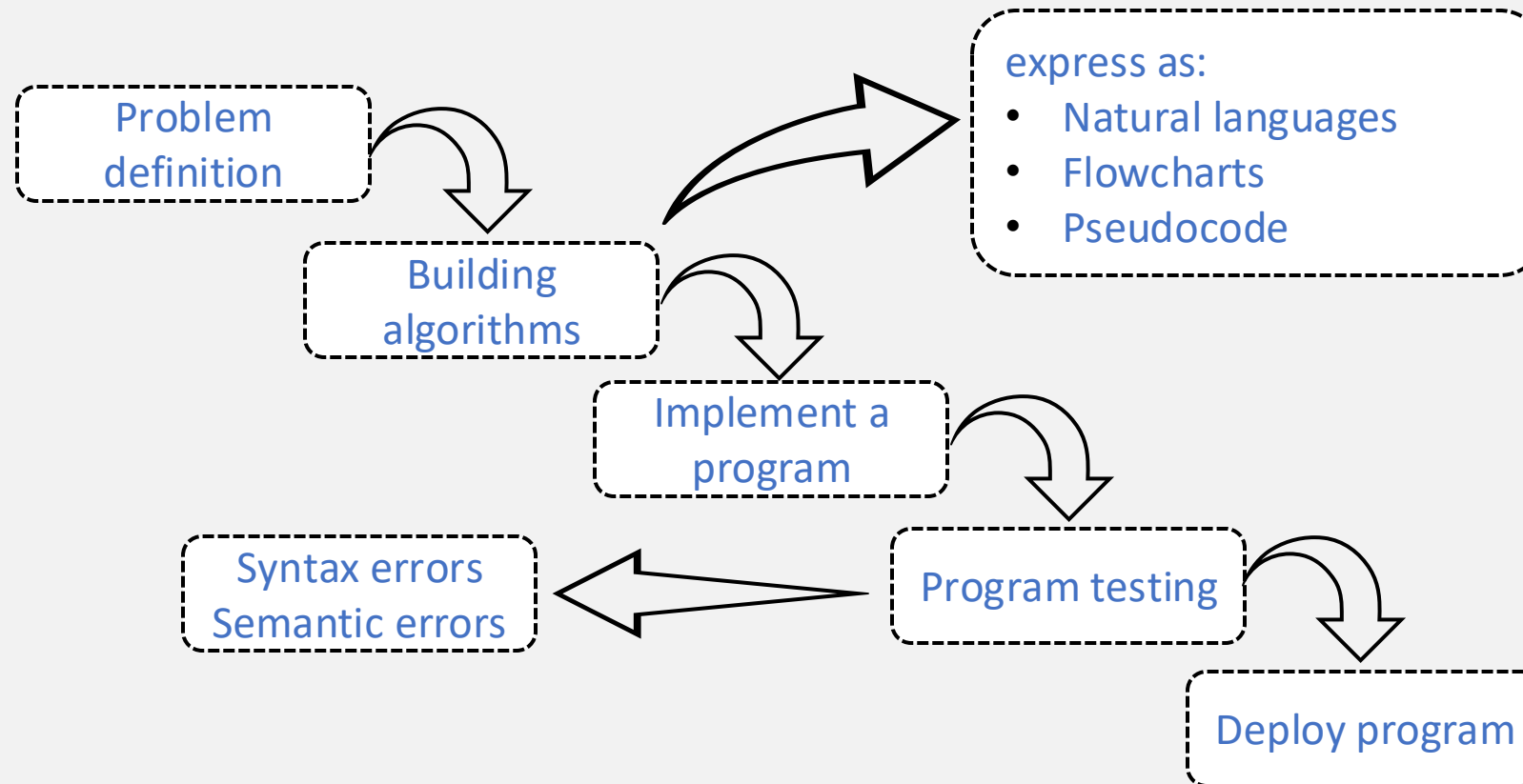
sources: [1](#), [2](#)

Write instructions to make a Peanut butter and Jelly sandwich



YouTube.com/JeshDarnit

Steps to build a program



Build an algorithm representation

- By natural languages
 - Limitation
 - Ambiguity
 - Less formal
- By flowcharts: Use defined shapes to express ideas
 - Easier to follow
- By pseudocode: borrow some simple languages
 - More formal
 - Not actual a programming language

Example

- Write an algorithm to solve a quadratic equation using representations:
 - By natural languages
 - By flowcharts
 - By pseudocode

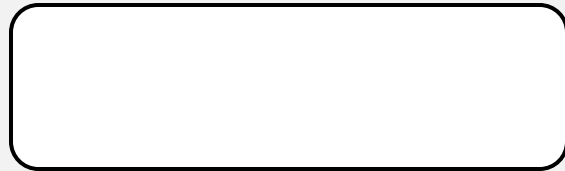
Natural languages

Input: a, b belong to real numbers

Output: solution of equation $ax + b = 0$

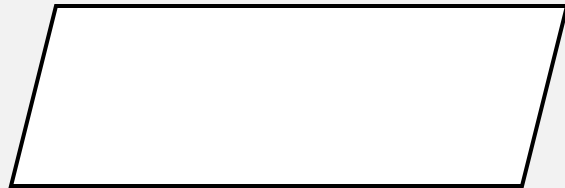
1. Input two real numbers a and b
2. If $a = 0$ then
 - 2.1. If $b = 0$ then
 - 2.1.1. Countless solutions
 - 2.1.2. End
 - 2.2. Else
 - 2.2.1. No solution
 - 2.2.2. End
3. Else
 - 3.1. There is one solution x
 - 3.2. The result $x = -b/a$
 - 3.3. End

Flowcharts



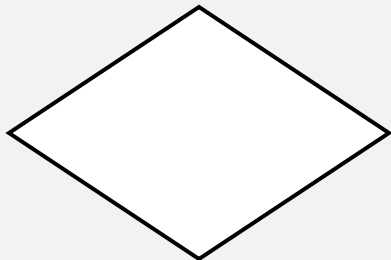
Alternate process

Indicate a starting and ending point



Input/Output operations

Indicate an input or output operation



Condition operation

Indicate a condition



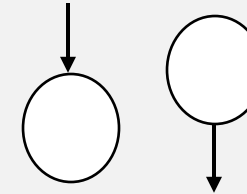
Process

Indicates a process



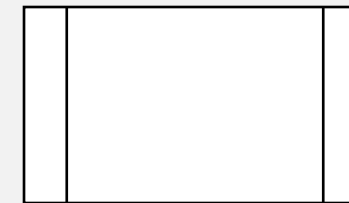
Flow

Indicate the next process



Connectors

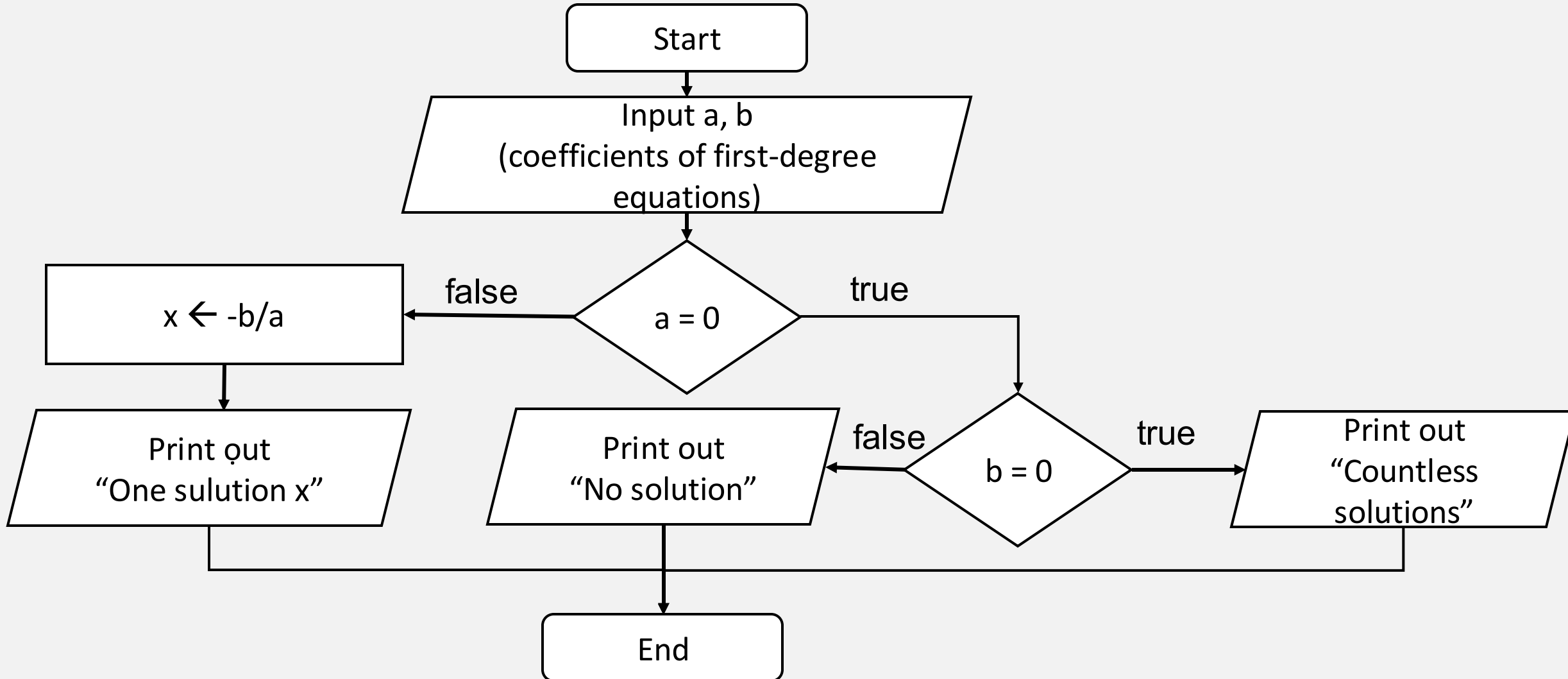
Connect segments of flowcharts



Modules

Defined functions

Flowcharts



Pseudocode

Input: a, b belong to $\mathbb{R}$

Output: solution of the equation $ax + b = 0$

if $a = 0$, then

begin

 if $b = 0$, then print "Countless solutions"

 else, print "No solution"

end

else

 print "There is one solution $x = -b/a$ "

Plan for today

- Introduction to Computers
- Introduction to Algorithms
- **Exercises**

Exercises

- 1) List one programming language that is an interpreted language and another that is a compiled language. What is the difference between interpreter and compiler? (> 3 criteria).
- 2) Find some examples about compile-time and running-time errors.
- 3) Given a number as a year, calculate the birthday.
- 4) Given a height and a weight, calculate the BMI score
- 5) Convert speed to pace, and vice versa.
- 6) Convert meter to mile, and vice versa.

Exercises

7) Given a year, check if it is a leap year?

8) Given scores and coefficients of three courses (e.g., such as Maths, Physics, Chemistry). Calculating the average score.

9) Given a vehicle plate contains 5 numbers (e.g., 12345), calculate the score of this plate.

Version 1: the last digit of sum of each digits, e.g.,

$1+2+3+4+5=15$ then output is 5 (points)

Version 2: the last digit of sum of each digits until the sum is less than 10, e.g.,

$1+2+3+4+5=15$, $1 + 5 = 6$ then output is 6 (points)

Exercises

- 10) Given three numbers, identify maximum and minimum of those numbers.
- 11) Given a student ID (8 numbers), identify numbers referring a admission year, faculty, type of program, and a sequence number.
- 12) Given a number, check if it is a prime number?